Version: Beta | Updated: 2026-06-17

AIStudio

AIStudio is a private, local AI search and RAG (Retrieval-Augmented Generation) application that runs entirely on your Mac. It lets you upload your own documents, index them, and ask questions in plain English — getting cited answers grounded in your content, with no data leaving your machine and no external API or cloud dependency.

AIStudio is what you use when you want to query your own documents with AI. It is not a general-purpose chatbot — it is a document intelligence system. You bring the documents; AIStudio finds the answers.

> *AIStudio is a hands-on AI engineering lab for exploring how LLM-enabled systems behave under real operational constraints — retrieval quality, vocabulary mismatch, metadata filtering, observability, and deployment trade-offs — before those issues get hidden behind abstractions.*
>
> ***Current stack:** Python · FastAPI · local Ollama inference · Qdrant vector storage · gemma3 · mistral · llama3.1 · nomic-embed-text · sentence-transformers · pdfplumber page-aware PDF extraction · CrossEncoder reranker · benchmark harness · CI/CD pipeline · Docker (future) · AWS ECS (future)*
>
> *The architecture choices are deliberate and documented. See docs/architecture_decisions.md. For a guide to how the codebase is organized, see docs/CODEBASE_GUIDE.md.*

Point of View

Agentic AI in Financial Services: Some Reflections

This work is one aspect of the author's work in AI. It explores the transition from descriptive to generative to agentic AI, the practical constraints on autonomous systems today, and a

framework for thinking about where AI adds durable value versus where human judgment remains irreplaceable. Written December 2025.

What This Does

AIStudio gives you a **private, local search engine over your own documents** — running entirely on your Mac, with no data leaving your machine and no external API dependency.

What you actually see and do

A browser-based interface lets you manage document collections and query them conversationally. Three main areas:

- **Corpus Manager** — create named collections of documents, upload PDFs, Word docs, PowerPoints, Excel files, or Markdown, and trigger indexing. Each corpus is independently searchable.
- **Chat Interface** — ask questions in plain English and get answers grounded in your documents, with inline source citations (`[1]` , `[2]`) and a References section showing exactly which document and passage each answer came from.
- **Filters** — optional firm and year filters narrow retrieval to a specific source before the query runs. Filtering happens at the system (vector) layer — not post-hoc on results.

Two corpora ship with AIStudio, each proving something different:

Demo corpus — 20 years of original thought leadership: AIStudio ships with a curated set of 9 documents spanning 2003–2026 — IT strategy frameworks, enterprise architecture methodology, financial services technology journals, cloud migration analysis, and AI reference architecture. These are original works: articles edited for practitioner journals, engagement frameworks, and strategy documents produced across senior technology roles at major financial institutions. Querying this corpus is querying the intellectual capital of a 20-year career. The corpus and the tool are the same proof point.

A reviewer who asks *"What is the relationship between business strategy and technology strategy?"* gets a grounded, cited answer from a 2006 FS Journal article — original work, not sample data. That is what makes the demo corpus distinctive.

AIStudio as its own corpus: AIStudio's documentation — architecture decisions, benchmark methodology, retrieval guides — is available as a corpus in the standard interface. Asking *"What embedding model does AIStudio use?"* or *"How does the reranker work?"* returns cited answers from the actual codebase docs. The tool documents itself.

On performance: Warm `llama3.1:70b` and warm `llama3.1:8b` are statistically identical in query latency on Apple Silicon (~6–7s average). Once loaded into unified memory, model size stops being a latency variable. See benchmarks/ for the full benchmark harness and timestamped reports.

The QUICKSTART also shows you how to set up the **SEC 10-K corpus** to demonstrate how AIStudio can operate at scale — **exploring 100+ annual filings from 21 financial services firms** (Goldman Sachs, JPMorgan Chase, Morgan Stanley, BlackRock, and others), 100K+ chunks **from over 900 MB of source filings**, ingested in roughly half an hour at ~54 chunks/sec on an M4 MacBook Pro. Due to their size, **the files for this corpus are not shipped with the app but need to be downloaded from the SEC first (utilities are provided)**. More importantly, **ingesting and indexing this type of corpus provides a good opportunity to learn how to work with a large corpus**.

ESEF corpus — European banks, the harder cousin: AIStudio also builds a second at-scale corpus from European banks' **ESEF** filings (retrieved from filings.xbrl.org by LEI — the same download → entities → glossary → ingest machinery as SEC, only the access key and endpoint change). The two corpora are deliberately complementary learning vehicles: together they surface the problems that actually bite in production — resolving the **many names one firm files under** (handled with a GLEIF/LEI entity knowledge base), **multilingual retrieval** (an English question against a filing that says *fonds propres de base de catégorie 1* rather than *Common Equity Tier 1* retrieves worse), and **pulling numbers out of dense, multi-column tables** without severing a cell from the year that gives it meaning. The Tutorial — Modules 2 and 3, plus Annexes 3 and 5 — walks through all of it end to end.

Models & benchmarking: AIStudio runs **any Ollama model** — `gemma3`, `mistral`, `llama3.1`, and whatever else you pull; `gemma3:4b` is the out-of-the-box default, the rest are options, not requirements. Model choice often barely moves warm latency (once weights are in unified memory, size stops being the variable), though it does change answer quality on the hard cases. Benchmarks here are normalized on the **Google Gemma suite** (`gemma3:27b`) for comparability — but the harness is model-agnostic: `ais_bench` automates question selection (by scope, topic, or id) and sweeps across models and retrieval parameters, so the same question set can run under permutations of model / α / top-k and be compared directly. See HARNESS.md.

Why this is hard (and how it's tested)

Nothing AIStudio does is rocket science these days — it's the table stakes of the job. You could assemble most of it on any cloud platform tomorrow. **What you can't do is point it at your *whole* corpus and freely mix your own files with outside sources.** A CISO might sign off on

dropping one file into Copilot; the whole corpus, blended with external feeds, is another matter — and in financial services that's the normal case, not the edge case.

AIStudio does what any retrieval system worth its salt must: **it verifies its own faithfulness at the level of the cited passage**, so a confident-sounding answer can be checked against the exact text it rests on.

The honest test bed. The system was audited claim-by-claim against the source chunks across a corpus of US and European bank filings. English is the clean regime, and retrieval fidelity degrades predictably with language — near-perfect on English, partial on French, weak on the Nordic/Italian/Dutch tail — so the results below are drawn from the English corpus. It handles three kinds of work:

- **Qualitative synthesis** — comparative questions across firms and years (how a set of banks describe cyber risk, climate exposure, regulatory burden, and how that shifts over successive filings), every claim tied back to its source passage. In the chunk-verified audit — ten questions, every claim checked, and a harness re-run rather than a one-off — it produced **zero fabricated figures**, abstaining where a fact wasn't retrievable rather than inventing one. One of the ten over-claimed against its own citation: a ratio label pulled from a genuinely complex table (the fix — a LlamaParse-class table extractor — is queued for the next build).
- **Numerical retrieval** — single-firm figures pulled from HTML/iXBRL tables, with a purpose-built extraction layer that binds each value to its column header and reporting year; audited figures verified exact to the reported precision.
- **Temporal evolution** — multi-year qualitative trends, with the system openly substituting the nearest available year rather than papering over a gap.

The (current) honest boundary, stated up front: qualitative synthesis across firms is established; the quantitative equivalent — assembling many firms' numbers into one correct comparison — is the open frontier. On the English corpus, where retrieval lands the right data cleanly, the failure root-causes to value-to-label binding at generation time — the model attaches a retrieved number to the wrong year or the wrong ratio type — which is downstream of retrieval and distinct from the language-driven retrieval gaps above.

AIStudio in numbers

Scale

- **Codebase** — ~28,400 lines of code across 95 files
- **SEC 10-K corpus** — 100+ filings · 21 firms · 901 MB of source filings · 100K+ chunks
- **ESEF corpus** — European bank annual reports, multilingual, retrieved by LEI

- **Demo corpus** — 9 original documents spanning 2003–2026
- **In-app help** — 15 reference documents

Performance *(MacBook Pro M4 Pro, 128 GB unified memory)*

- **Ingest** — ~54 chunks/sec · 100K+ chunks in ~31 min · 0 failures
- **Warm query** — ~6–7 s (llama3.1 8b and 70b statistically identical)
- **SEC 10-K synthesis** — ~58 s avg (gemma3:27b, multi-firm)
- **Retrieval** — ~0.3–0.5 s even at 100K+ chunks
- **Benchmark** — demo 14/14 (gemma3:27b, K=10) · 12/14 (llama3.1:8b, K=5) · SEC 10-K 10/10 mechanical, 9/10 audited (gemma3:27b) · 8/10 mechanical, 9/10 substantive (llama3.1:8b)

Documentation

AIStudio ships ~15 reference documents (about 80 pages). Start wherever fits what you're doing:

Document	What it covers	Read it when…
README	Product overview, point of view, architecture, benchmarks	You're deciding what AIStudio is
about	One-page overview shown in the UI About panel	You clicked About or want the gist
QUICKSTART	Install + first run in under 30 min	You're setting it up
HOWTO	Corpora, upload, filters, query settings, troubleshooting	You're using it day to day
TUTORIAL	Guided walkthroughs + SEC 10-K at-scale + benchmarking	You want to go deep
architecture_elements	How the pieces fit + data flow (the mental model)	You want the "under the hood" picture
architecture_decisions	Why Qdrant / CrossEncoder / chunking	You're weighing the technical choices
api_introduction		

Document	What it covers	Read it when…
	The local HTTP API — retrieve vs ask, firm isolation	You're building an integration
CODEBASE_GUIDE	Directory layout, files, the ingest/query pipeline	You're reading or extending the code
FILE_GUIDE	Commands, files, services reference	You need to look up a command or file
DEMO_CORPUS	What ships in the demo + suggested questions	You're exploring the demo
HARNESS	Running benchmarks, CLI flags, reading reports	You're measuring quality
QA_TESTING_LESSONS_LEARNED	Install friction + QA findings	You hit a snag or want the honest record
dependencies	Python + system dependency versions	You're troubleshooting the environment
PRODUCT_ROADMAP	What works now and the direction beyond Beta	You want to know what's next

Quickstart

See QUICKSTART.md to get a running instance in under 30 minutes.

TL;DR for experienced users:

```
# 1. Install Qdrant (not in Homebrew — binary install required)
curl -L https://github.com/qdrant/qdrant/releases/latest/download/qdrant-aarch64-apple-darw
mkdir -p ~/bin && mv qdrant ~/bin/qdrant && echo 'export PATH="$HOME/bin:$PATH"' >> ~/.zshr

# 2. Clone and set up
mkdir -p ~/Developer && cd ~/Developer
git clone https://github.com/mbarberony/AIStudio.git && cd AIStudio
./ais_install

# 3. Start everything (stops any running services first, then restarts clean)
ais_start

# 4. Open UI
open front_end/rag_studio.html
```

Architecture — Local (Current)

[Architecture diagram]

Why Qdrant over ChromaDB:

	ChromaDB	Qdrant
Stability at scale	Crashed at 32K chunks	Stable at 100K+ chunks
Language	Python	Rust
Metadata filtering	Limited	Native Filter/FieldCondition
Memory model	Python GC	Rust ownership, near-zero overhead
Production path	Single-node	Sharding, replication, quantization
gRPC	No	Yes (port 6334)

Architecture — Cloud (Future)

The local four-process pattern maps directly to a containerized cloud deployment. Each process becomes a container; Qdrant and Ollama have official Docker images.

[Architecture diagram]

Local → cloud (a future release) is a container boundary, not an architecture change. Same FastAPI app, same Qdrant queries, same Ollama interface — wrapped in Docker and deployed to ECS Fargate. No code changes required.

Performance Findings

Synthesized from benchmark runs on MacBook Pro M4 Pro (128GB unified memory):

- **8.6s avg latency** per query at α=0.5 hybrid retrieval, K=10 — including complex multi-source synthesis

- **14/14 pass rate** on demo corpus benchmark with `gemma3:27b`, K=10, α=0.5 hybrid retrieval (M4 Pro, 128GB unified memory). With `llama3.1:8b` at K=5: **12/14 mechanical, 13/14 substantive**. Questions file updated to v2.2.0 (2026-06-16) — prior versions had keyword brittleness preventing 14/14 on any model.
- **10/10 mechanical · 9/10 audited** on the curated SEC 10-K question set (`gemma3:27b`, 10 cross-firm questions, 21 firms, 100K+ chunks). With `llama3.1:8b`: **8/10 mechanical · 9/10 substantive** — the 9/10 substantive is consistent across both models. Precise table-cell extraction is the known frontier; see audited reports under `benchmarks/sec_10k/reports/`
- **SEC 10-K synthesis runs ~58s avg** on `gemma3:27b` — long, multi-firm answers, so output-token generation dominates (consistent with the bottleneck below), not retrieval
- **Model size does not predict warm latency** — llama3.1:70b and llama3.1:8b both land at ~6s warm; the bottleneck is output token generation, not parameter count
- **Retrieval adds ~0.3–0.5s** even at 100K+ chunks — inference, not retrieval, is the bottleneck
- **Stable across successive runs** — no thermal throttling or memory pressure observed

All figures from Beast (M4 Pro, 128GB unified memory). MacBook Air (M4) clean install validated — latency is approximately 4–5× higher at equivalent load, consistent with the memory bandwidth differential between M4 Pro and M4 base. Demo corpus results use hybrid retrieval (α=0.5, K=10); pure vector retrieval (default) achieves 13/14.

→ Benchmark reports and question sets

Benchmark

```
Corpus:     100+ SEC 10-K filings, 21 financial services firms
Chunks:     100K+
Ingest:     ~31 min, ~54 chunks/sec, 0 failures
Model:      gemma3:4b (default) · gemma3:27b (SEC 10-K / benchmark) · llama3.1 / mistral (a
Latency:    ~58s avg on the SEC 10-K cross-firm synthesis set (gemma3:27b);
            ~6s warm on demo (llama3.1 8b/70b identical, M4 Pro 128GB)
Benchmark:  SEC 10-K 10/10 mech · 9/10 audited (gemma3:27b); 8/10 mech · 9/10 substantive (
            demo 14/14 (gemma3:27b, K=10); 12/14 mech · 13/14 substantive (llama3.1:8b, K=5
Frontier:   precise table-cell extraction — see benchmarks/sec_10k/reports/ (audited)
ChromaDB:   crashed at 32,285 chunks
Qdrant:     stable at 100K+ chunks
```

See benchmarks/ for the full benchmark harness, timestamped reports, and question sets.

Current Status

Core RAG loop working end-to-end on a 100K-chunk production corpus. Qdrant vector store (Rust-based) replaced ChromaDB after ChromaDB crashed at 32K chunks. Metadata filtering (firm, year) implemented end-to-end — backend, API, and UI. CI/CD pipeline green on every push.

Working today

- Document ingestion: PDF, Word, PowerPoint, Excel, Markdown, HTML
- Vector search via Qdrant 1.17.0 with cosine similarity
- Metadata filtering — firm and year filters at the vector layer
- RAG query with inline citations and source references
- Browser UI — corpus management (create, rename, delete, stats, inspect), file upload with progress, chat with citations
- Corpus rename — renames directory, cascades corpus_metadata.yaml, triggers background re-index
- Stats panel — per-file KB sizes, last ingestion time and duration from corpus_metadata.yaml
- Auto-linkify — URLs and corpus filenames in responses rendered as clickable links
- FastAPI backend — `/ask`, `/health`, `/corpus/*` (create, rename, delete, info, upload, ingest), `/source`, `/prewarm`
- Auto-launch script — `ais_start` starts all three services (Qdrant, Ollama, FastAPI backend)
- Benchmark harness — `benchmarks/bench.py` with CLI flags, auto-generates findings
- CI/CD — GitHub Actions: lint + unit + integration tests on every push
- Developer tooling — Makefile (`make check`, `make coverage`), pre-commit hooks

Recently completed (Beta)

- CrossEncoder reranker (ms-marco-MiniLM) — fixes vocabulary mismatch ✅
- Page-aware PDF chunking via pdfplumber — page numbers in citations ✅
- `Open ↗` links — click any citation reference to open the source file directly in your browser ✅
- Hybrid retrieval (M2.A) — `Retrieval Mix` slider blends vector semantic search with BM25 keyword matching per query; α=0 pure semantic, α=1 pure keyword, default 0.5 ✅
- `--force` ingest flag — atomic wipe + clean re-index ✅
- YAML benchmark question files with corpus auto-detection ✅
- Demo corpus benchmark: 14/14 questions pass at α=0.5 hybrid retrieval, K=10, gemma3:27b (M4 Pro) ✅ · 12/14 with llama3.1:8b at K=5

- Corpus rename — UI + API, background re-index, re-ingest time estimate ✅
- Ingestion metadata — last_ingested_at and duration persisted to corpus_metadata.yaml ✅

- Manifest-driven `ais_install` — adds any new command alias in one step ✅
- Help corpus search guidance — per-document routing rules injected into system prompt ✅

- Relevance threshold — low-scoring chunks discarded at the vector layer ✅
- XBRL noise stripping in HTML ingestion ✅

What's next: the Post-Beta and Future phases — Source Dive (citation → exact page), a one-click `.dmg` installer, published API docs, Docker + AWS ECS Fargate, GPU inference — live in PRODUCT_ROADMAP.md.

> *AIStudio is intentionally in a state of permanent Beta — not as a disclaimer, but as a design principle: the proof point is a living system, always being improved, never declared finished. Versioning marks milestones, not completion.*

Manuel Barbero · mbarberony@gmail.com · linkedin.com/in/mbarberony · github.com/mbarberony/AIStudio